

Playing Atari with Deep Reinforcement Learning

DQN 2013 中文基础注释版

原作者：Volodymyr Mnih 等
注释稿：基于作者公开 PDF 编写

原论文：2013 年 12 月；注释版生成于 2026 年 7 月

这篇论文的历史位置

这是通常被视为首次提出 **Deep Q-Network (DQN)** 的论文。核心贡献不是单独发明 Q-learning、卷积网络或经验回放，而是把它们组合成一个能从 Atari 原始像素端到端学习控制策略的系统，并用同一套算法和超参数处理多个游戏。2015 年 Nature 论文是更成熟、规模更大的后续版本。

阅读方式

每章先给出原论文页缩略图定位，再用中文拆解概念、公式、算法和实验。本文是学习型注释，不是逐句翻译；原论文仍是最终依据。建议先读“零基础预备”，再按页阅读。

阅读地图

1 零基础预备：论文到底在解决什么	4
1.1 把打游戏写成一个学习问题	4
1.2 策略、价值与 Q 值	4
1.3 Bellman 方程：把长期问题拆成一步	4
1.4 为什么要用深度网络	5
2 第 1 页：问题、动机与贡献	5
2.1 摘要逐层拆解	6
2.2 为什么当时很难	6
3 第 2 页：强化学习背景	7
3.1 MDP 假设是什么	7
3.2 Q-learning 更新从哪里来	8
3.3 on-policy 与 off-policy	8
4 第 3 页：从理论到深度 Q 网络	9
4.1 神经网络版本的损失	9
4.2 为什么相关样本会伤害训练	10
5 第 4 页：算法 1 与网络输入	11
5.1 算法逐行翻译成可执行思路	11
5.2 终止状态为什么没有未来项	12
5.3 画面预处理与帧堆叠	12
6 第 5 页：网络结构与实验设计	13
6.1 三层结构怎么读	13
6.2 奖励裁剪的意义	14
6.3 固定超参数为何重要	14
7 第 6 页：训练曲线与 Q 值诊断	15
7.1 为什么 episode reward 抖动	15
7.2 论文的固定验证状态集	16
7.3 Seaquest 可视化说明什么	16
8 第 7 页：结果表怎么读	17
8.1 基线的含义	17
8.2 训练分数与测试分数	18
8.3 为什么不同游戏分数不能直接平均	18

9 第 8–9 页：结论、局限与文献	18
9.1 作者能合理声称什么	19
9.2 2013 版本的主要局限	19
9.3 值得顺藤摸瓜的文献线	20
10 把算法完整串起来	21
10.1 一条样本的数值例子	21
10.2 “致命三元组”视角	21
11 2013 与 2015 版本差异速查	21
12 术语表	21
13 读完后的自测题与答案	22

1 零基础预备：论文到底在解决什么

1.1 把打游戏写成一个学习问题

游戏每一时刻给出屏幕画面，智能体选择动作，环境返回下一帧画面和分数变化。可以把一次交互写成

$$s_t \xrightarrow{a_t} (r_t, s_{t+1}),$$

其中 s_t 是时刻 t 的状态， a_t 是动作， r_t 是即时奖励。目标不是只拿到眼前奖励，而是让未来累计奖励尽可能大：

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

$\gamma \in [0, 1)$ 是折扣因子 (discount factor)。它越接近 1，智能体越重视长远结果；越接近 0，越“短视”。

直觉例子

在 Breakout 中，移动挡板本身通常没有奖励，若只看 r_t ，智能体不知道这一动作是否有用。累计回报把“现在移动挡板”和“几秒后接住球、打掉砖块”连接起来，这就是信用分配 (credit assignment) 问题。

1.2 策略、价值与 Q 值

策略 (policy) 记为 $\pi(a | s)$ ，表示在状态 s 下选择动作 a 的规则或概率。动作价值函数 (action-value function) 定义为：在状态 s 先做动作 a ，之后继续按某个策略行动时，预期能得到多少累计回报。

$$Q^\pi(s, a) = \mathbb{E}_\pi[R_t | s_t = s, a_t = a].$$

最优动作价值 $Q^*(s, a)$ 假设之后每一步都采取最优行动。知道 Q^* 后，控制问题变得简单：

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

换句话说，DQN 的神经网络不是直接输出“向左”或“向右”，而是为所有可选动作各输出一个分数，再选最大的那个。

1.3 Bellman 方程：把长期问题拆成一步

最优 Q 值满足递归关系：

$$Q^*(s_t, a_t) = \mathbb{E} \left[r_t + \gamma \max_{a'} Q^*(s_{t+1}, a') \right].$$

右边由“眼前奖励”与“下一状态最有价值的动作”组成。这条方程是 DQN 训练目标的来源。若网络当前预测为 $Q(s_t, a_t; \theta)$ ，一个自然的目标值是

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta).$$

然后让预测 $Q(s_t, a_t; \theta)$ 接近 y_t 。 θ 表示神经网络的全部参数。

初学者最容易混淆的三件事

- r_t 是一次转移的即时奖励； R_t 是从现在开始的累计回报。
- $Q(s, a)$ 评价的是“状态与动作的组合”，不是只评价状态。
- $\max_{a'}$ 用于构造学习目标；真正执行动作时仍需要探索，不能永远只取最大值。

1.4 为什么要用深度网络

Atari 一帧是大量像素。若把每一种画面都当作独立状态，状态空间大得不可枚举。卷积神经网络通过共享局部滤波器学习“边缘、物体、运动线索”等特征，再把它们映射到每个动作的 Q 值。这叫**函数逼近 (function approximation)**：用 $Q(s, a; \theta)$ 近似无法存表的 $Q^*(s, a)$ 。

自检

若某状态下网络输出 $[1.2, 0.3, 2.1]$ ，三个分量对应左、停、右，那么贪心动作是什么？答案：右，因为其预测 Q 值最大。注意这不是动作概率。

2 第 1 页：问题、动机与贡献

Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou

Daan Wierstra Martin Riedmiller

DeepMind Technologies

{vlad, koray, david, alex.graves, ioannis, daan, martin.riedmiller} @ deepmind.com

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.

1 Introduction

Learning to control agents directly from high-dimensional sensory inputs like vision and speech is one of the long-standing challenges of reinforcement learning (RL). Most successful RL applications that operate on these domains have relied on hand-crafted features combined with linear value functions or policy representations. Clearly, the performance of such systems heavily relies on the quality of the feature representation.

Recent advances in deep learning have made it possible to extract high-level features from raw sensory data, leading to breakthroughs in computer vision [11, 22, 16] and speech recognition [6, 7]. These methods utilise a range of neural network architectures, including convolutional networks, multilayer perceptrons, restricted Boltzmann machines and recurrent neural networks, and have exploited both supervised and unsupervised learning. It seems natural to ask whether similar techniques could also be beneficial for RL with sensory data.

However reinforcement learning presents several challenges from a deep learning perspective. Firstly, most successful deep learning applications to date have required large amounts of hand-labelled training data. RL algorithms, on the other hand, must be able to learn from a scalar reward signal that is frequently sparse, noisy and delayed. The delay between actions and resulting rewards, which can be thousands of timesteps long, seems particularly daunting when compared to the direct association between inputs and targets found in supervised learning. Another issue is that most deep learning algorithms assume the data samples to be independent, while in reinforcement learning one typically encounters sequences of highly correlated states. Furthermore, in RL the data distribution changes as the algorithm learns new behaviours, which can be problematic for deep learning methods that assume a fixed underlying distribution.

This paper demonstrates that a convolutional neural network can overcome these challenges to learn successful control policies from raw video data in complex RL environments. The network is trained with a variant of the Q-learning [26] algorithm, with stochastic gradient descent to update the weights. To alleviate the problems of correlated data and non-stationary distributions, we use

本页先记住

论文想证明：单个通用智能体可以只看像素、只接收分数，用同一套学习方法掌握多个不同的 **Atari** 游戏。

2.1 摘要逐层拆解

摘要中有四个限定词很重要：

1. **raw pixels**: 输入不是人工提取的物体位置，而是经过简单预处理的画面。
2. **model-free**: 算法不学习游戏的显式转移模型，不先预测“按左键后画面会怎样”。
3. **end-to-end**: 从视觉输入到动作价值的特征由最终控制目标共同训练。
4. **same architecture**: 不同游戏尽量共用网络设计和超参数，强调通用性。

2.2 为什么当时很难

深度学习训练通常假设样本近似独立同分布，而强化学习产生的数据有三个麻烦：连续帧高度相关；策略变化导致数据分布不断变化；网络预测出的 Q 值又参与构造自己的训练目标。这三个因素叠加，容易造成振荡甚至发散。DQN 的关键工程思想是用**经验回放**（**experience replay**）缓和前两个问题。

“发明 DQN” 应准确理解为

Q-learning 来自 1989 年，卷积网络更早，经验回放也有前史。2013 论文的原创性在于把深度卷积表示与 off-policy Q-learning、经验回放整合为可工作的 Atari 控制系统，并给出跨游戏实证。不要把它表述成“发明了强化学习”或“发明了 Q-learning”。

3 第 2 页：强化学习背景



Figure 1: Screen shots from five Atari 2600 Games: (Left-to-right) Pong, Breakout, Space Invaders, Seaquest, Beam Rider

an experience replay mechanism [13] which randomly samples previous transitions, and thereby smooths the training distribution over many past behaviors.

We apply our approach to a range of Atari 2600 games implemented in The Arcade Learning Environment (ALE) [3]. Atari 2600 is a challenging RL testbed that presents agents with a high dimensional visual input (210×160 RGB video at 60Hz) and a diverse and interesting set of tasks that were designed to be difficult for humans players. Our goal is to create a single neural network agent that is able to successfully learn to play as many of the games as possible. The network was not provided with any game-specific information or hand-designed visual features, and was not privy to the internal state of the emulator; it learned from nothing but the video input, the reward and terminal signals, and the set of possible actions—just as a human player would. Furthermore the network architecture and all hyperparameters used for training were kept constant across the games. So far the network has outperformed all previous RL algorithms on six of the seven games we have attempted and surpassed an expert human player on three of them. Figure 1 provides sample screenshots from five of the games used for training.

2 Background

We consider tasks in which an agent interacts with an environment $\mathcal{E}$, in this case the Atari emulator, in a sequence of actions, observations and rewards. At each time-step the agent selects an action a_t from the set of legal game actions, $\mathcal{A} = \{1, \dots, K\}$. The action is passed to the emulator and modifies its internal state and the game score. In general $\mathcal{E}$ may be stochastic. The emulator's internal state is not observed by the agent; instead it observes an image $x_t \in \mathbb{R}^d$ from the emulator, which is a vector of raw pixel values representing the current screen. In addition it receives a reward r_t representing the change in game score. Note that in general the game score may depend on the whole prior sequence of actions and observations; feedback about an action may only be received after many thousands of time-steps have elapsed.

Since the agent only observes images of the current screen, the task is partially observed and many emulator states are perceptually aliased, i.e. it is impossible to fully understand the current situation from only the current screen x_t . We therefore consider sequences of actions and observations, $s_t = x_1, a_1, x_2, \dots, a_{t-1}, x_t$, and learn game strategies that depend upon these sequences. All sequences in the emulator are assumed to terminate in a finite number of time-steps. This formalism gives rise to a large but finite Markov decision process (MDP) in which each sequence is a distinct state. As a result, we can apply standard reinforcement learning methods for MDPs, simply by using the complete sequence s_t as the state representation at time t .

The goal of the agent is to interact with the emulator by selecting actions in a way that maximises future rewards. We make the standard assumption that future rewards are discounted by a factor of γ per time-step, and define the future discounted return at time t as $R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$, where T is the time-step at which the game terminates. We define the optimal action-value function $Q^*(s, a)$ as the maximum expected return achievable by following any strategy, after seeing some sequence s and then taking some action a , $Q^*(s, a) = \max_{\pi} \mathbb{E}[R_t | s_t = s, a_t = a, \pi]$, where π is a policy mapping sequences to actions (or distributions over actions).

The optimal action-value function obeys an important identity known as the Bellman equation. This is based on the following intuition: if the optimal value $Q^*(s', a')$ of the sequence s' at the next time-step was known for all possible actions a' , then the optimal strategy is to select the action a'

2

原论文第 2 页缩略图，仅用于定位下文注释。

本页先记住

这一页建立 MDP、回报、最优 Q 函数和 Bellman 方程，并说明传统表格 Q-learning 为什么不能直接处理像素。

3.1 MDP 假设是什么

马尔可夫决策过程 (Markov Decision Process, MDP) 要求：给定当前状态和动作，下一状态的分布不再依赖更早历史。形式上，

$$P(s_{t+1} | s_t, a_t, s_{t-1}, \dots) = P(s_{t+1} | s_t, a_t).$$

单张 Atari 画面通常不满足这一点，因为看不出球的速度方向。因此论文把若干连续帧组成状态。这是一个实用近似：帧堆叠能显露运动，但仍不保证完整可观测。

3.2 Q-learning 更新从哪里来

表格 Q-learning 的经典更新是

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[\underbrace{r_t + \gamma \max_{a'} Q(s_{t+1}, a')}_{\text{TD 目标}} - \underbrace{Q(s_t, a_t)}_{\text{当前预测}} \right].$$

方括号里的差称为时序差分误差 (temporal-difference error, TD error):

$$\delta_t = y_t - Q(s_t, a_t).$$

若 $\delta_t > 0$, 说明结果比预期好, 应提高该状态动作的估值; 若小于 0, 则应降低。

3.3 on-policy 与 off-policy

Q-learning 是离策略 (off-policy) 算法: 行为策略可以用 ϵ -greedy 探索, 但更新目标使用 $\max_{a'} Q(s', a')$, 学习的是贪心目标策略。 ϵ -greedy 的规则是:

$$a_t = \begin{cases} \text{随机动作}, & \text{概率 } \epsilon, \\ \arg \max_a Q(s_t, a; \theta), & \text{概率 } 1 - \epsilon. \end{cases}$$

训练早期 ϵ 较大以收集多样经验, 之后逐渐减小以更多利用已学知识。

自检

为什么不能把 ϵ 立即设为 0? 因为初始 Q 值几乎是随机的, 纯贪心会反复执行偶然占优的动作, 无法发现更好的行为。

4 第 3 页：从理论到深度 Q 网络

maximising the expected value of $r + \gamma Q^*(s', a')$,

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (1)$$

The basic idea behind many reinforcement learning algorithms is to estimate the action-value function, by using the Bellman equation as an iterative update, $Q_{i+1}(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q_i(s', a') \mid s, a]$. Such *value iteration* algorithms converge to the optimal action-value function, $Q_i \rightarrow Q^*$ as $i \rightarrow \infty$ [23]. In practice, this basic approach is totally impractical, because the action-value function is estimated separately for each sequence, without any generalisation. Instead, it is common to use a function approximator to estimate the action-value function, $Q(s, a; \theta) \approx Q^*(s, a)$. In the reinforcement learning community this is typically a linear function approximator, but sometimes a non-linear function approximator is used instead, such as a neural network. We refer to a neural network function approximator with weights θ as a Q-network. A Q-network can be trained by minimising a sequence of loss functions $L_i(\theta_i)$ that changes at each iteration i ,

$$L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} \left[(y_i - Q(s, a; \theta_i))^2 \right], \quad (2)$$

where $y_i = \mathbb{E}_{s' \sim \mathcal{E}} [r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) \mid s, a]$ is the target for iteration i and $\rho(s, a)$ is a probability distribution over sequences s and actions a that we refer to as the *behaviour distribution*. The parameters from the previous iteration θ_{i-1} are held fixed when optimising the loss function $L_i(\theta_i)$. Note that the targets depend on the network weights; this is in contrast with the targets used for supervised learning, which are fixed before learning begins. Differentiating the loss function with respect to the weights we arrive at the following gradient,

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot); s' \sim \mathcal{E}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right]. \quad (3)$$

Rather than computing the full expectations in the above gradient, it is often computationally expedient to optimise the loss function by stochastic gradient descent. If the weights are updated after every time-step, and the expectations are replaced by single samples from the behaviour distribution ρ and the emulator $\mathcal{E}$ respectively, then we arrive at the familiar *Q-learning* algorithm [26].

Note that this algorithm is *model-free*: it solves the reinforcement learning task directly using samples from the emulator $\mathcal{E}$, without explicitly constructing an estimate of $\mathcal{E}$. It is also *off-policy*: it learns about the greedy strategy $a = \max_a Q(s, a; \theta)$, while following a behaviour distribution that ensures adequate exploration of the state space. In practice, the behaviour distribution is often selected by an ϵ -greedy strategy that follows the greedy strategy with probability $1 - \epsilon$ and selects a random action with probability ϵ .

3 Related Work

Perhaps the best-known success story of reinforcement learning is *TD-gammon*, a backgammon-playing program which learnt entirely by reinforcement learning and self-play, and achieved a super-human level of play [24]. TD-gammon used a model-free reinforcement learning algorithm similar to Q-learning, and approximated the value function using a multi-layer perceptron with one hidden layer¹.

However, early attempts to follow up on TD-gammon, including applications of the same method to chess, Go and checkers were less successful. This led to a widespread belief that the TD-gammon approach was a special case that only worked in backgammon, perhaps because the stochasticity in the dice rolls helps explore the state space and also makes the value function particularly smooth [19].

Furthermore, it was shown that combining model-free reinforcement learning algorithms such as Q-learning with non-linear function approximators [25], or indeed with off-policy learning [1] could cause the Q-network to diverge. Subsequently, the majority of work in reinforcement learning focused on linear function approximators with better convergence guarantees [25].

¹In fact TD-Gammon approximated the state value function $V(s)$ rather than the action-value function $Q(s, a)$, and learnt *on-policy* directly from the self-play games

原论文第 3 页缩略图，仅用于定位下文注释。

本页先记住

真正的困难不是写出 Bellman 方程，而是让一个不断变化的数据生成过程稳定地训练深度网络。

4.1 神经网络版本的损失

论文把第 i 次迭代的损失写为

$$L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} \left[(y_i - Q(s, a; \theta_i))^2 \right], \quad y_i = \mathbb{E}_{s'} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) \right].$$

它看起来像普通回归：输入是状态，监督标签是 y_i 。但标签不是人工提供，而是由当前或较早的网络估计出来，所以叫自举（bootstrapping）。

对参数求梯度，忽略目标对当前参数的反向传播，可得

$$\nabla_{\theta_i} L_i = \mathbb{E} [2 (Q(s, a; \theta_i) - y_i) \nabla_{\theta_i} Q(s, a; \theta_i)] .$$

反向传播只更新被执行动作 a 对应的输出分支，但共享的卷积特征会受到影响。

4.2 为什么相关样本会伤害训练

如果按时间顺序训练，连续画面几乎一样，一个 minibatch 可能全是同一局面。梯度方向高度偏向局部轨迹，且一次参数更新又会改变后续收集数据的策略。经验回放把转移

$$e_t = (s_t, a_t, r_t, s_{t+1})$$

放入缓冲区 D ，训练时随机抽取。这样可以：

- 打乱时间相关性，使 minibatch 更接近独立样本；
- 重复利用过去经验，提高数据效率；
- 混合由不同阶段策略产生的数据，减缓分布骤变。

经验回放不是什么

它不是回放游戏视频，也不是让智能体真的回到过去。缓冲区保存的是数值化转移，随机抽样用于梯度更新。普通均匀回放也不保证样本完全独立，只是显著降低相邻相关性。

5 第 4 页: 算法 1 与网络输入

More recently, there has been a revival of interest in combining deep learning with reinforcement learning. Deep neural networks have been used to estimate the environment $\mathcal{E}$; restricted Boltzmann machines have been used to estimate the value function [21]; or the policy [9]. In addition, the divergence issues with Q-learning have been partially addressed by *gradient temporal-difference* methods. These methods are proven to converge when evaluating a fixed policy with a nonlinear function approximator [14]; or when learning a control policy with linear function approximation using a restricted variant of Q-learning [15]. However, these methods have not yet been extended to nonlinear control.

Perhaps the most similar prior work to our own approach is neural fitted Q-learning (NFQ) [20]. NFQ optimises the sequence of loss functions in Equation 2, using the RPROP algorithm to update the parameters of the Q-network. However, it uses a batch update that has a computational cost per iteration that is proportional to the size of the data set, whereas we consider stochastic gradient updates that have a low constant cost per iteration and scale to large data-sets. NFQ has also been successfully applied to simple real-world control tasks using purely visual input, by first using deep autoencoders to learn a low dimensional representation of the task, and then applying NFQ to this representation [12]. In contrast our approach applies reinforcement learning end-to-end, directly from the visual inputs; as a result it may learn features that are directly relevant to discriminating action-values. Q-learning has also previously been combined with experience replay and a simple neural network [13], but again starting with a low-dimensional state rather than raw visual inputs.

The use of the Atari 2600 emulator as a reinforcement learning platform was introduced by [3], who applied standard reinforcement learning algorithms with linear function approximation and generic visual features. Subsequently, results were improved by using a larger number of features, and using tug-of-war hashing to randomly project the features into a lower-dimensional space [2]. The HyperNEAT evolutionary architecture [8] has also been applied to the Atari platform, where it was used to evolve (separately, for each distinct game) a neural network representing a strategy for that game. When trained repeatedly against deterministic sequences using the emulator's reset facility, these strategies were able to exploit design flaws in several Atari games.

4 Deep Reinforcement Learning

Recent breakthroughs in computer vision and speech recognition have relied on efficiently training deep neural networks on very large training sets. The most successful approaches are trained directly from the raw inputs, using lightweight updates based on stochastic gradient descent. By feeding sufficient data into deep neural networks, it is often possible to learn better representations than handcrafted features [11]. These successes motivate our approach to reinforcement learning. Our goal is to connect a reinforcement learning algorithm to a deep neural network which operates directly on RGB images and efficiently process training data by using stochastic gradient updates.

Tesauro's TD-Gammon architecture provides a starting point for such an approach. This architecture updates the parameters of a network that estimates the value function, directly from on-policy samples of experience, $s_t, a_t, r_t, s_{t+1}, a_{t+1}$, drawn from the algorithm's interactions with the environment (or by self-play, in the case of backgammon). Since this approach was able to outperform the best human backgammon players 20 years ago, it is natural to wonder whether two decades of hardware improvements, coupled with modern deep neural network architectures and scalable RL algorithms might produce significant progress.

In contrast to TD-Gammon and similar online approaches, we utilize a technique known as *experience replay* [13] where we store the agent's experiences at each time-step, $e_t = (s_t, a_t, r_t, s_{t+1})$ in a data-set $\mathcal{D} = e_1, \dots, e_N$, pooled over many episodes into a *replay memory*. During the inner loop of the algorithm, we apply Q-learning updates, or minibatch updates, to samples of experience, $e \sim \mathcal{D}$, drawn at random from the pool of stored samples. After performing experience replay, the agent selects and executes an action according to an ϵ -greedy policy. Since using histories of arbitrary length as inputs to a neural network can be difficult, our Q-function instead works on fixed length representation of histories produced by a function ϕ . The full algorithm, which we call *deep Q-learning*, is presented in Algorithm 1.

This approach has several advantages over standard online Q-learning [23]. First, each step of experience is potentially used in many weight updates, which allows for greater data efficiency.

4

原论文第 4 页缩略图, 仅用于定位下文注释。

本页先记住

DQN 是“与环境交互一步, 存入经验, 再随机抽一批经验做一次 Q-learning 更新”的循环。

5.1 算法逐行翻译成可执行思路

1. 初始化容量有限的回放缓冲区 D , 随机初始化网络参数 θ 。
2. 每个 episode 开始时获得初始画面, 并建立预处理后的状态序列。
3. 用 ϵ -greedy 选动作: 随机探索或选择当前 Q 值最大的动作。
4. 执行动作, 观察奖励与下一画面, 把新画面并入状态。
5. 把 (s_t, a_t, r_t, s_{t+1}) 存入 D 。

6. 从 D 均匀随机抽取 minibatch。
7. 若转移终止，目标 $y_j = r_j$ ；否则 $y_j = r_j + \gamma \max_{a'} Q(s_{j+1}, a'; \theta)$ 。
8. 对 $(y_j - Q(s_j, a_j; \theta))^2$ 做一步梯度下降。

5.2 终止状态为什么没有未来项

游戏结束后不存在 s_{t+1} 的可实现未来回报，因此

$$y_t = \begin{cases} r_t, & \text{terminal,} \\ r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta), & \text{otherwise.} \end{cases}$$

若错误地在终止后继续 bootstrap，会把不存在的价值加到最后一步，产生系统偏差。

5.3 画面预处理与帧堆叠

论文将 Atari 画面转灰度、缩放到 84×84 ，并把最近 4 帧堆叠作为网络输入。帧堆叠不是为了提高图像清晰度，而是提供速度方向线索。单帧看到球在左边，无法判断它正在向左还是向右；连续帧可以。

输入张量的形状

一个状态可理解为 $84 \times 84 \times 4$ 。前两个维度是空间位置，最后一维是时间上相邻的四个灰度帧。现代框架常采用 $4 \times 84 \times 84$ 的通道优先布局，语义相同。

6 第 5 页：网络结构与实验设计

Algorithm 1 Deep Q-learning with Experience Replay

```

Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
Initialize action-value function  $Q$  with random weights
for episode = 1,  $M$  do
  Initialise sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
  for  $t = 1, T$  do
    With probability  $\epsilon$  select a random action  $a_t$ 
    otherwise select  $a_t = \max_a Q^*(\phi(s_t), a; \theta)$ 
    Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$ 
    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$ 
    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  according to equation 3
  end for
end for

```

Second, learning directly from consecutive samples is inefficient, due to the strong correlations between the samples; randomizing the samples breaks these correlations and therefore reduces the variance of the updates. Third, when learning on-policy the current parameters determine the next data sample that the parameters are trained on. For example, if the maximizing action is to move left then the training samples will be dominated by samples from the left-hand side; if the maximizing action then switches to the right then the training distribution will also switch. It is easy to see how unwanted feedback loops may arise and the parameters could get stuck in a poor local minimum, or even diverge catastrophically [25]. By using experience replay the behavior distribution is averaged over many of its previous states, smoothing out learning and avoiding oscillations or divergence in the parameters. Note that when learning by experience replay, it is necessary to learn off-policy (because our current parameters are different to those used to generate the sample), which motivates the choice of Q-learning.

In practice, our algorithm only stores the last N experience tuples in the replay memory, and samples uniformly at random from $\mathcal{D}$ when performing updates. This approach is in some respects limited since the memory buffer does not differentiate important transitions and always overwrites with recent transitions due to the finite memory size N . Similarly, the uniform sampling gives equal importance to all transitions in the replay memory. A more sophisticated sampling strategy might emphasize transitions from which we can learn the most, similar to prioritized sweeping [17].

4.1 Preprocessing and Model Architecture

Working directly with raw Atari frames, which are 210×160 pixel images with a 128 color palette, can be computationally demanding, so we apply a basic preprocessing step aimed at reducing the input dimensionality. The raw frames are preprocessed by first converting their RGB representation to gray-scale and down-sampling it to a 110×84 image. The final input representation is obtained by cropping an 84×84 region of the image that roughly captures the playing area. The final cropping stage is only required because we use the GPU implementation of 2D convolutions from [11], which expects square inputs. For the experiments in this paper, the function ϕ from algorithm 1 applies this preprocessing to the last 4 frames of a history and stacks them to produce the input to the Q -function.

There are several possible ways of parameterizing Q using a neural network. Since Q maps history-action pairs to scalar estimates of their Q-value, the history and the action have been used as inputs to the neural network by some previous approaches [20, 12]. The main drawback of this type of architecture is that a separate forward pass is required to compute the Q-value of each action, resulting in a cost that scales linearly with the number of actions. We instead use an architecture in which there is a separate output unit for each possible action, and only the state representation is an input to the neural network. The outputs correspond to the predicted Q-values of the individual action for the input state. The main advantage of this type of architecture is the ability to compute Q-values for all possible actions in a given state with only a single forward pass through the network.

5

原论文第 5 页缩略图，仅用于定位下文注释。

本页先记住

网络把一组图像帧映射为固定数量的动作价值；同一个卷积骨干服务所有动作输出。

6.1 三层结构怎么读

2013 版网络主要由两层卷积和一层全连接隐藏层构成，输出层每个单元对应一个合法动作。若某游戏有 6 个动作，网络输出就是长度 6 的向量：

$$f_{\theta}(s) = [Q(s, a_1), \dots, Q(s, a_6)].$$

一次前向传播同时得到所有动作价值，比“每个动作单独跑一次网络”高效。

卷积层的局部连接与权重共享适合图像：同一个球或砖块可能出现在不同位置，滤波器无需为每个位置重新学习。全连接层再把全局空间关系组合成动作决策。

6.2 奖励裁剪的意义

论文把正奖励裁为 +1、负奖励裁为 -1、零保持 0。这样不同游戏的分数尺度被标准化，同一学习率更容易跨游戏使用。代价是丢掉奖励幅度信息：得到 100 分和 1 分在训练信号上可能相同。因此它是一种强调稳定性与通用性的工程折中。

6.3 固定超参数为何重要

如果每个游戏都精细调参，结果可能只说明研究者熟悉这些游戏。论文尽量使用统一设置，测试的是算法能否适应不同视觉与控制规律。唯一自然变化的是输出动作数，因为不同游戏合法动作集合不同。

7 第 6 页：训练曲线与 Q 值诊断

We now describe the exact architecture used for all seven Atari games. The input to the neural network consists is an $84 \times 84 \times 4$ image produced by ϕ . The first hidden layer convolves $16 \times 8 \times 8$ filters with stride 4 with the input image and applies a rectifier nonlinearity [10, 18]. The second hidden layer convolves $32 \times 4 \times 4$ filters with stride 2, again followed by a rectifier nonlinearity. The final hidden layer is fully-connected and consists of 256 rectifier units. The output layer is a fully-connected linear layer with a single output for each valid action. The number of valid actions varied between 4 and 18 on the games we considered. We refer to convolutional networks trained with our approach as Deep Q-Networks (DQN).

5 Experiments

So far, we have performed experiments on seven popular ATARI games – Beam Rider, Breakout, Enduro, Pong, Q*bert, Seaquest, Space Invaders. We use the same network architecture, learning algorithm and hyperparameters settings across all seven games, showing that our approach is robust enough to work on a variety of games without incorporating game-specific information. While we evaluated our agents on the real and unmodified games, we made one change to the reward structure of the games during training only. Since the scale of scores varies greatly from game to game, we fixed all positive rewards to be 1 and all negative rewards to be -1 , leaving 0 rewards unchanged. Clipping the rewards in this manner limits the scale of the error derivatives and makes it easier to use the same learning rate across multiple games. At the same time, it could affect the performance of our agent since it cannot differentiate between rewards of different magnitude.

In these experiments, we used the RMSProp algorithm with minibatches of size 32. The behavior policy during training was ϵ -greedy with ϵ annealed linearly from 1 to 0.1 over the first million frames, and fixed at 0.1 thereafter. We trained for a total of 10 million frames and used a replay memory of one million most recent frames.

Following previous approaches to playing Atari games, we also use a simple frame-skipping technique [3]. More precisely, the agent sees and selects actions on every k^{th} frame instead of every frame, and its last action is repeated on skipped frames. Since running the emulator forward for one step requires much less computation than having the agent select an action, this technique allows the agent to play roughly k times more games without significantly increasing the runtime. We use $k = 4$ for all games except Space Invaders where we noticed that using $k = 4$ makes the lasers invisible because of the period at which they blink. We used $k = 3$ to make the lasers visible and this change was the only difference in hyperparameter values between any of the games.

5.1 Training and Stability

In supervised learning, one can easily track the performance of a model during training by evaluating it on the training and validation sets. In reinforcement learning, however, accurately evaluating the progress of an agent during training can be challenging. Since our evaluation metric, as suggested by [3], is the total reward the agent collects in an episode or game averaged over a number of games, we periodically compute it during training. The average total reward metric tends to be very noisy because small changes to the weights of a policy can lead to large changes in the distribution of states the policy visits. The leftmost two plots in figure 2 show how the average total reward evolves during training on the games Seaquest and Breakout. Both averaged reward plots are indeed quite noisy, giving one the impression that the learning algorithm is not making steady progress. Another, more stable, metric is the policy's estimated action-value function Q , which provides an estimate of how much discounted reward the agent can obtain by following its policy from any given state. We collect a fixed set of states by running a random policy before training starts and track the average of the maximum² predicted Q for these states. The two rightmost plots in figure 2 show that average predicted Q increases much more smoothly than the average total reward obtained by the agent and plotting the same metrics on the other five games produces similarly smooth curves. In addition to seeing relatively smooth improvement to predicted Q during training we did not experience any divergence issues in any of our experiments. This suggests that, despite lacking any theoretical convergence guarantees, our method is able to train large neural networks using a reinforcement learning signal and stochastic gradient descent in a stable manner.

²The maximum for each state is taken over the possible actions.

原论文第 6 页缩略图，仅用于定位下文注释。

本页先记住

奖励曲线噪声很大；固定状态集上的平均最大 Q 值是更平滑的学习进度指标，但它也可能过度乐观。

7.1 为什么 episode reward 抖动

同一策略可能因为随机初始状态、探索动作和游戏动力学得到差异很大的分数。短窗口平均仍会剧烈波动，所以不能看到一次分数下降就断言训练退化。应关注较长趋势，并使用独立评估阶段减少探索噪声。

7.2 论文的固定验证状态集

作者从训练前收集的一组状态上计算 $\max_a Q(s, a)$ 的平均值。因为状态集合不变，曲线主要反映网络估值变化。若该值平滑上升，通常表示网络认为自己能取得更高未来回报。

Q 值上升不等于真的变强

Q-learning 中的最大化操作可能产生**过估计 (overestimation)**。如果预测 Q 值上升而真实评估分数不升，可能只是网络越来越自信。必须把价值估计与实际游戏得分一起看。后来 Double DQN 专门缓解这一问题。

7.3 Seaquest 可视化说明什么

论文展示一小段轨迹中预测价值的变化。智能体在即将获得奖励之前就提高估值，说明网络学到某些事件前兆，而非只在奖励出现后机械响应。可视化是解释模型行为的线索，不是严格的因果证明。

8 第 7 页：结果表怎么读

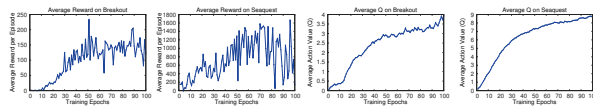


Figure 2: The two plots on the left show average reward per episode on Breakout and Seaquest respectively during training. The statistics were computed by running an ϵ -greedy policy with $\epsilon = 0.05$ for 10000 steps. The two plots on the right show the average maximum predicted action-value of a held out set of states on Breakout and Seaquest respectively. One epoch corresponds to 50000 minibatch weight updates or roughly 30 minutes of training time.

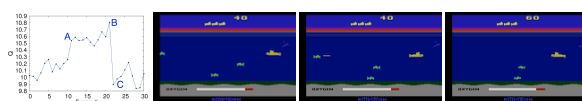


Figure 3: The leftmost plot shows the predicted value function for a 30 frame segment of the game Seaquest. The three screenshots correspond to the frames labeled by A, B, and C respectively.

5.2 Visualizing the Value Function

Figure 3 shows a visualization of the learned value function on the game Seaquest. The figure shows that the predicted value jumps after an enemy appears on the left of the screen (point A). The agent then fires a torpedo at the enemy and the predicted value peaks as the torpedo is about to hit the enemy (point B). Finally, the value falls to roughly its original value after the enemy disappears (point C). Figure 3 demonstrates that our method is able to learn how the value function evolves for a reasonably complex sequence of events.

5.3 Main Evaluation

We compare our results with the best performing methods from the RL literature [3, 4]. The method labeled **Sarsa** used the Sarsa algorithm to learn linear policies on several different feature sets hand-engineered for the Atari task and we report the score for the best performing feature set [3]. **Contingency** used the same basic approach as **Sarsa** but augmented the feature sets with a learned representation of the parts of the screen that are under the agent's control [4]. Note that both of these methods incorporate significant prior knowledge about the visual problem by using background subtraction and treating each of the 128 colors as a separate channel. Since many of the Atari games use one distinct color for each type of object, treating each color as a separate channel can be similar to producing a separate binary map encoding the presence of each object type. In contrast, our agents only receive the raw RGB screenshots as input and must *learn* to detect objects on their own.

In addition to the learned agents, we also report scores for an expert human game player and a policy that selects actions uniformly at random. The human performance is the median reward achieved after around two hours of playing each game. Note that our reported human scores are much higher than the ones in Bellemare et al. [3]. For the learned methods, we follow the evaluation strategy used in Bellemare et al. [3, 5] and report the average score obtained by running an ϵ -greedy policy with $\epsilon = 0.05$ for a fixed number of steps. The first five rows of table 1 show the per-game average scores on all games. Our approach (labeled DQN) outperforms the other learning methods by a substantial margin on all seven games despite incorporating almost no prior knowledge about the inputs.

We also include a comparison to the evolutionary policy search approach from [8] in the last three rows of table 1. We report two sets of results for this method. The **HNeat Best** score reflects the results obtained by using a hand-engineered object detector algorithm that outputs the locations and

原论文第 7 页缩略图，仅用于定位下文注释。

本页先记住

DQN 在 7 个游戏中的 6 个超过此前最佳方法，并在部分游戏接近或超过人类；但测试规模仍小。

8.1 基线的含义

论文比较了随机策略、线性函数逼近、若干既有强化学习方法以及人类玩家。公平比较时要确认输入信息、训练帧数、是否使用游戏专属特征和评估协议是否一致。表格中的一个数字并不能消除协议差异。

8.2 训练分数与测试分数

训练阶段含探索，测试阶段通常使用较小但非零的 ϵ ，并在固定时长内评估。测试分数更接近策略能力，但依然受随机性影响。理想报告应给出多随机种子均值和方差；2013 论文的统计完整性按今天标准较有限。

8.3 为什么不同游戏分数不能直接平均

Pong 的分数范围与 Breakout、Seaquest 完全不同，原始分数平均会被数值尺度大的游戏支配。后来常用人类归一化分数：

$$\text{normalized} = \frac{\text{agent} - \text{random}}{\text{human} - \text{random}}.$$

0 表示随机水平，1 表示所选人类基准水平。它也有缺点：若人类与随机差距很小，分母会放大噪声；“人类水平”还依赖具体玩家和评测时长。

9 第 8–9 页：结论、局限与文献

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	-20.4	157	110	179
Sarsa [3]	996	5.2	129	-19	614	665	271
Contingency [4]	1743	6	159	-17	960	723	268
DQN	4092	168	470	20	1952	1705	581
Human	7456	31	368	-3	18900	28010	3690
HNeat Best [8]	3616	52	106	19	1800	920	1720
HNeat Pixel [8]	1332	4	91	-16	1325	800	1145
DQN Best	5184	225	661	21	4500	1740	1075

Table 1: The upper table compares average total reward for various learning methods by running an ϵ -greedy policy with $\epsilon = 0.05$ for a fixed number of steps. The lower table reports results of the single best performing episode for HNeat and DQN. HNeat produces deterministic policies that always get the same score while DQN used an ϵ -greedy policy with $\epsilon = 0.05$.

types of objects on the Atari screen. The **HNeat Pixel** score is obtained by using the special 8 color channel representation of the Atari emulator that represents an object label map at each channel. This method relies heavily on finding a deterministic sequence of states that represents a successful exploit. It is unlikely that strategies learnt in this way will generalize to random perturbations; therefore the algorithm was only evaluated on the highest scoring single episode. In contrast, our algorithm is evaluated on ϵ -greedy control sequences, and must therefore generalize across a wide variety of possible situations. Nevertheless, we show that on all the games, except Space Invaders, not only our max evaluation results (row 8), but also our average results (row 4) achieve better performance.

Finally, we show that our method achieves better performance than an expert human player on Breakout, Enduro and Pong and it achieves close to human performance on Beam Rider. The games Q*bert, Seaquest, Space Invaders, on which we are far from human performance, are more challenging because they require the network to find a strategy that extends over long time scales.

6 Conclusion

This paper introduced a new deep learning model for reinforcement learning, and demonstrated its ability to master difficult control policies for Atari 2600 computer games, using only raw pixels as input. We also presented a variant of online Q-learning that combines stochastic minibatch updates with experience replay memory to ease the training of deep networks for RL. Our approach gave state-of-the-art results in six of the seven games it was tested on, with no adjustment of the architecture or hyperparameters.

References

- [1] Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the 12th International Conference on Machine Learning (ICML 1995)*, pages 30–37. Morgan Kaufmann, 1995.
- [2] Marc Bellemare, Joel Veness, and Michael Bowling. Sketch-based linear value function approximation. In *Advances in Neural Information Processing Systems 25*, pages 2222–2230, 2012.
- [3] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.
- [4] Marc G Bellemare, Joel Veness, and Michael Bowling. Investigating contingency awareness using atari 2600 games. In *AAAI*, 2012.
- [5] Marc G. Bellemare, Joel Veness, and Michael Bowling. Bayesian learning of recursively factored environments. In *Proceedings of the Thirtieth International Conference on Machine Learning (ICML 2013)*, pages 1211–1219, 2013.

原论文第 8 页缩略图，仅用于定位下文注释。

本页先记住

结论强调通用性；参考文献显示 DQN 是多个成熟思想的组合创新。

9.1 作者能合理声称什么

论文证明确实存在一种统一方法，可从高维视觉输入学习多个控制任务，并明显优于当时若干基线。它没有证明算法适用于所有游戏，也没有证明获得了通用智能。7 个游戏是重要概念验证，但覆盖面有限。

9.2 2013 版本的主要局限

- 只测试 7 个 Atari 游戏，代表性有限。
- 样本效率低，需要大量环境帧；人类通常用少得多的经验学习。
- 网络只接收短帧堆叠，处理长期记忆的能力弱。
- 同一网络既选择又评估最大动作，存在过估计。
- 没有 2015 Nature 版明确使用的独立 target network 稳定机制。
- 结果对随机种子与实现细节可能敏感，论文未按现代标准充分报告不确定性。

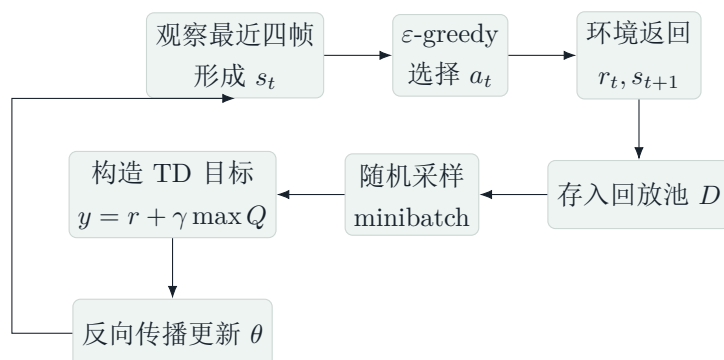
- [6] George E. Dahl, Dong Yu, Li Deng, and Alex Acero. Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition. *Audio, Speech, and Language Processing, IEEE Transactions on*, 20(1):30–42, January 2012.
- [7] Alex Graves, Abdel-rahman Mohamed, and Geoffrey E. Hinton. Speech recognition with deep recurrent neural networks. In *Proc. ICASSP*, 2013.
- [8] Matthew Hausknecht, Risto Miikkulainen, and Peter Stone. A neuro-evolution approach to general atari game playing. 2013.
- [9] Nicolas Heess, David Silver, and Yee Whye Teh. Actor-critic reinforcement learning with energy-based policies. In *European Workshop on Reinforcement Learning*, page 43, 2012.
- [10] Kevin Jarrett, Koray Kavukcuoglu, Marc'Aurelio Ranzato, and Yann LeCun. What is the best multi-stage architecture for object recognition? In *Proc. International Conference on Computer Vision and Pattern Recognition (CVPR 2009)*, pages 2146–2153. IEEE, 2009.
- [11] Alex Krizhevsky, Ilya Sutskever, and Geoff Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25*, pages 1106–1114, 2012.
- [12] Sascha Lange and Martin Riedmiller. Deep auto-encoder neural networks in reinforcement learning. In *Neural Networks (IJCNN), The 2010 International Joint Conference on*, pages 1–8. IEEE, 2010.
- [13] Long-Ji Lin. Reinforcement learning for robots using neural networks. Technical report, DTIC Document, 1993.
- [14] Hamid Maei, Csaba Szepesvári, Shalabh Bhatnagar, Doina Precup, David Silver, and Rich Sutton. Convergent Temporal-Difference Learning with Arbitrary Smooth Function Approximation. In *Advances in Neural Information Processing Systems 22*, pages 1204–1212, 2009.
- [15] Hamid Maei, Csaba Szepesvári, Shalabh Bhatnagar, and Richard S. Sutton. Toward off-policy learning control with function approximation. In *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, pages 719–726, 2010.
- [16] Volodymyr Mnih. *Machine Learning for Aerial Image Labeling*. PhD thesis, University of Toronto, 2013.
- [17] Andrew Moore and Chris Atkeson. Prioritized sweeping: Reinforcement learning with less data and less real time. *Machine Learning*, 13:103–130, 1993.
- [18] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, pages 807–814, 2010.
- [19] Jordan B. Pollack and Alan D. Blair. Why did td-gammon work. In *Advances in Neural Information Processing Systems 9*, pages 10–16, 1996.
- [20] Martin Riedmiller. Neural fitted q iteration—first experiences with a data efficient neural reinforcement learning method. In *Machine Learning: ECML 2005*, pages 317–328. Springer, 2005.
- [21] Brian Sallans and Geoffrey E. Hinton. Reinforcement learning with factored states and actions. *Journal of Machine Learning Research*, 5:1063–1088, 2004.
- [22] Pierre Sermanet, Koray Kavukcuoglu, Soumith Chintala, and Yann LeCun. Pedestrian detection with unsupervised multi-stage feature learning. In *Proc. International Conference on Computer Vision and Pattern Recognition (CVPR 2013)*. IEEE, 2013.
- [23] Richard Sutton and Andrew Barto. *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- [24] Gerald Tesauro. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- [25] John N Tsitsiklis and Benjamin Van Roy. An analysis of temporal-difference learning with function approximation. *Automatic Control, IEEE Transactions on*, 42(5):674–690, 1997.
- [26] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3-4):279–292, 1992.

原论文第 9 页缩略图，仅用于定位下文注释。

9.3 值得顺藤摸瓜的文献线

主题	建议理解的历史关系
Q-learning	Watkins 与 Dayan：无模型、离策略的时序差分控制。
卷积网络	LeCun 等：局部连接与权重共享带来视觉归纳偏置。
经验回放	过去经验的随机再利用，为在线学习数据去相关。
TD-Gammon	神经网络与时序差分学习结合的早期成功案例，但输入与任务更专门。
2015 Nature DQN	增加 target network、扩大到 49 个游戏、引入更系统的人类归一化评估。

10 把算法完整串起来



10.1 一条样本的数值例子

设某转移奖励 $r = 1$, $\gamma = 0.99$, 下一状态各动作预测为 $[2.0, 3.5, 1.0]$, 当前动作预测为 2.4。则

$$y = 1 + 0.99 \times 3.5 = 4.465, \quad \delta = y - Q = 2.065.$$

平方误差为 $2.065^2 \approx 4.264$ 。梯度下降会推动当前动作的预测从 2.4 向 4.465 靠近。注意其他动作的输出不是这条样本的直接回归目标, 但共享网络参数会使它们间接受影响。

10.2 “致命三元组”视角

后来强化学习教材常用三个因素解释不稳定: 函数逼近、自举、离策略学习。DQN 三者齐全, 所以不能假设普通监督学习的稳定性。经验回放缓和数据相关和分布漂移, 却不从理论上消除发散风险; 2015 版的 target network 进一步减慢目标变化。

11 2013 与 2015 版本差异速查

维度	2013 首次提出版	2015 Nature 成熟版
游戏数量	7 个	49 个
网络	两个卷积层, 规模较小	三个卷积层, 容量更大
稳定化	经验回放为核心; 目标与在线网络耦合较强	经验回放 + 独立 target network 周期复制
训练规模	概念验证规模	约 2 亿帧/游戏的系统实验
评估	与若干基线、人类比较	多种归一化、固定起始与随机 no-op 协议、消融
历史意义	首次把系统命名并展示为 DQN 路线	让 DQN 成为深度强化学习的标志性结果

12 术语表

术语	基础解释
agent / 智能体	作出动作并从奖励中学习的系统。
environment / 环境	接收动作，产生下一观察和奖励的外部系统。
state / 状态	决策所依据的信息；论文中用四帧预处理画面近似。
action / 动作	Atari 控制器允许的离散按键组合。
reward / 奖励	环境单步返回的标量反馈。
return / 回报	从某时刻开始的折扣累计奖励。
Q-value / 动作价值	在状态中先做某动作后的预期长期回报。
TD target / 时序差分目标	$r + \gamma \max Q(s', a')$ ，由一步真实奖励和下一步估值构成。
bootstrap / 自举	用已有估计构造新的学习目标。
experience replay / 经验回放	保存转移并随机重采样训练。
minibatch / 小批量	一次梯度更新共同使用的一组样本。
episode / 回合	从游戏开始到终止的一段交互。
exploration / 探索	尝试当前看起来不一定最优的动作以获取信息。
exploitation / 利用	选择当前估值最高的动作。
off-policy / 离策略	收集数据的策略与被学习的目标策略可以不同。

13 读完后的自测题与答案

1. 为什么输入四帧而不是一帧？单帧缺少速度和方向，四帧让状态更接近马尔可夫。
2. 经验回放解决什么？降低连续样本相关性、重复利用经验、平滑数据分布变化。
3. DQN 输出是不是动作概率？不是，是每个动作的预测累计回报；执行时再用 ϵ -greedy 选动作。
4. TD 目标为何会移动？它包含网络自身对下一状态的估计，参数变化会改变标签。
5. 奖励裁剪有什么代价？稳定跨游戏尺度，但丢失奖励大小的细粒度差别。
6. 为何称端到端？卷积特征不是另行标注训练，而是由最终 Q-learning 损失共同学习。
7. 2013 版与 2015 版最关键的算法差异？2015 版明确采用独立 target network 并周期更新，使 TD 目标更稳定。

一句话总结

DQN 2013 的核心是：用卷积网络近似最优动作价值函数，用经验回放打散在线轨迹，再用 Q-learning 的 Bellman 自举目标做端到端训练，从而把“看像素”和“选动作”连成一个通用系统。

版本说明

本注释稿基于作者公开的 9 页论文 PDF 编写。它提供教学性释义、推导和批判性阅读提示，不替代原文，也不代表原作者或出版机构。公式符号尽量与原文一致；为初学者清晰起见，部分期望和索引写法作了简化。